STA 336: Statistical Machine Learning

Homework 7

Rento Saijo

April 16, 2026

Table of Contents

Disclosure **2**

Problem 1 **3**
- Problem 1 (1) . . . 4
- Problem 1 (2) . . . 5
- Problem 1 (3) . . . 6
- Problem 1 (4) . . . 7
- Problem 1 (5) . . . 8
- Problem 1 (6) . . . 10
- Problem 1 (7) . . . 11
- Problem 1 (8) . . . 13
- Problem 1 (9) . . . 15
- Problem 1 (10) . . . 17
- Problem 1 (11a) . . . 18
- Problem 1 (11b) . . . 19
- Problem 1 (11c) . . . 22
- Problem 1 (11d) . . . 25

Problem 2 **27**
- Problem 2 (a) . . . 28
- Problem 2 (b) . . . 30

Problem 3 **32**
- Problem 3 (a) . . . 33
- Problem 3 (b) . . . 34
- Problem 3 (c) . . . 36
- Problem 3 (d) . . . 37
- Problem 3 (e) . . . 38
- Problem 3 (f) . . . 39
- Problem 3 (g) . . . 40
- Problem 3 (h) . . . 41
- Problem 3 (i) . . . 42
- Problem 3 (j) . . . 45
- Problem 3 (k) . . . 47

Disclosure

GPT-5.4 Codex was used to create the `YAML` portion and some `LaTeX` code to format the text/equations nicely. Page formatting code was also provided by Derin Gezgin. In the setup chunk, helper functions such as `table_latex()`, `confusion_df()`, and `test_mse()` were defined so the tree-based analyses can be reported clearly and reproducibly. See the original `RMD` file here for more details.

Problem 1

Problem 1

Using the `Auto` data, answer the following questions on classification trees.

For final tree-regression subproblem on `Sales`, I use `Carseats` data from `ISLR2`, since `Sales` is quantitative response in that data set.

Problem 1 (1)

Problem 1 (1)

Define a categorical variable `ECO` based on `mpg`. If `mpg` is greater than median, call it `Economy`; otherwise, call it `Consuming`.

Cell 1

```r
# Load Auto, define ECO, check median, first few values.
auto_df <- ISLR2::Auto
auto_median_mpg <- median(auto_df$mpg)
auto_df$ECO <- factor(
  ifelse(auto_df$mpg > auto_median_mpg, 'Economy', 'Consuming'),
  levels = c('Consuming', 'Economy')
)
auto_median_mpg
```

```
## [1] 22.75
```

Cell 2

```r
head(auto_df[, c('mpg', 'ECO')])
```

```
##   mpg       ECO
## 1  18 Consuming
## 2  15 Consuming
## 3  18 Consuming
## 4  16 Consuming
## 5  17 Consuming
## 6  15 Consuming
```

Using the median `mpg` value `median(mpg)` $= 22.75$, I defined

$$\texttt{ECO} = \begin{cases} \texttt{Economy}, & \text{if } \texttt{mpg} > 22.75, \\ \texttt{Consuming}, & \text{if } \texttt{mpg} \leq 22.75. \end{cases}$$

So `ECO` is a perfectly balanced two-level categorical variable determined directly by whether a car is above or below the median fuel efficiency.

Problem 1 (2)

Problem 1 (2)

Check number of observations for each group of `ECO`.

Cell 3

```r
# Count observations in each ECO group.
eco_count_tbl <- tibble::tibble(
  ECO = names(table(auto_df$ECO)),
  observations = as.integer(table(auto_df$ECO))
)

# Print ECO counts.
eco_count_tbl %>%
  table_latex(
    col_names = c('ECO Group', 'Observations'),
    caption   = 'Number of observations in each ECO category.'
  )
```

Table 1: Number of observations in each ECO category.

ECO Group	Observations
Consuming	196
Economy	196

There are 196 observations in `Consuming` and 196 observations in `Economy`. This happens because the split is based on the sample median, so the data are divided evenly into the two groups.

Problem 1 (3)

Problem 1 (3)

Build a classification tree on `ECO` with all variables except `name`, and interpret it.

Cell 4

```r
# Fit classification tree using all predictors except name.
auto_tree_full <- tree::tree(ECO ~ . - name, data = auto_df)
auto_tree_full
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
## 1) root 392 543.4 Consuming ( 0.5 0.5 )
##   2) mpg < 22.75 196   0.0 Consuming ( 1.0 0.0 ) *
##   3) mpg > 22.75 196   0.0 Economy ( 0.0 1.0 ) *
```

Cell 5

```r
summary(auto_tree_full)
```

```
##
## Classification tree:
## tree::tree(formula = ECO ~ . - name, data = auto_df)
## Variables actually used in tree construction:
## [1] "mpg"
## Number of terminal nodes:  2
## Residual mean deviance:  0 = 0 / 390
## Misclassification error rate: 0 = 0 / 392
```

The fitted tree is extremely simple: it makes a single split at `mpg < 22.75`. Cars with `mpg < 22.75` are classified as `Consuming`, while cars with `mpg > 22.75` are classified as `Economy`. The training misclassification error is 0, because the response `ECO` was defined directly from `mpg`, so once the tree is allowed to use `mpg`, it can recover the rule exactly.

Problem 1 (4)

Problem 1 (4)

What is the single variable that is used for the tree? Why do you think it is used? Is this tree good?

The only variable used is `mpg`. It is used because `ECO` was literally constructed from whether `mpg` is above or below the median, so `mpg` contains the exact information needed to separate the two classes. In a narrow predictive sense, the tree is perfect on this data set, since its training error is $0/392 = 0$. However, it is not especially interesting scientifically, because it is just reproducing the rule used to define the response in the first place. So it is a perfect but tautological tree.

Problem 1 (5)

Problem 1 (5)

Exclude `mpg`. Predict `ECO` using only `horsepower`, `weight`, and `acceleration`, and show tree with text.

Cell 6

```r
# Fit reduced classification tree without mpg.
auto_tree_reduced <- tree::tree(ECO ~ horsepower + weight + acceleration, data =
auto_df)
auto_tree_reduced
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##  1) root 392 543.400 Consuming ( 0.50000 0.50000 )
##    2) weight < 2764.5 191 123.700 Economy ( 0.09948 0.90052 )
##      4) weight < 2224.5 98  11.160 Economy ( 0.01020 0.98980 )
##        8) horsepower < 87 92   0.000 Economy ( 0.00000 1.00000 ) *
##        9) horsepower > 87 6   5.407 Economy ( 0.16667 0.83333 ) *
##      5) weight > 2224.5 93  91.390 Economy ( 0.19355 0.80645 ) *
##    3) weight > 2764.5 201 147.000 Consuming ( 0.88060 0.11940 )
##      6) horsepower < 93.5 35  48.260 Consuming ( 0.54286 0.45714 )
##       12) weight < 2961 8   6.028 Economy ( 0.12500 0.87500 ) *
##       13) weight > 2961 27  34.370 Consuming ( 0.66667 0.33333 )
##         26) acceleration < 19.55 17  12.320 Consuming ( 0.88235 0.11765 )
##           52) weight < 3018 5   6.730 Consuming ( 0.60000 0.40000 ) *
##           53) weight > 3018 12   0.000 Consuming ( 1.00000 0.00000 ) *
##         27) acceleration > 19.55 10  12.220 Economy ( 0.30000 0.70000 )
##           54) weight < 3260 5   0.000 Economy ( 0.00000 1.00000 ) *
##           55) weight > 3260 5   6.730 Consuming ( 0.60000 0.40000 ) *
##      7) horsepower > 93.5 166  64.130 Consuming ( 0.95181 0.04819 )
##       14) weight < 2953.5 21  25.130 Consuming ( 0.71429 0.28571 )
##         28) acceleration < 14.45 5   5.004 Economy ( 0.20000 0.80000 ) *
##         29) acceleration > 14.45 16  12.060 Consuming ( 0.87500 0.12500 ) *
##       15) weight > 2953.5 145  21.110 Consuming ( 0.98621 0.01379 )
##         30) acceleration < 17.3 128   0.000 Consuming ( 1.00000 0.00000 ) *
##         31) acceleration > 17.3 17  12.320 Consuming ( 0.88235 0.11765 ) *
```

Cell 7

```r
# Plot reduced tree with smaller text.
with_family({
  par(mar = c(1.5, 1.5, 1.5, 1.5))
  plot(auto_tree_reduced)
  text(auto_tree_reduced, pretty = 0, cex = 0.8)
})
```

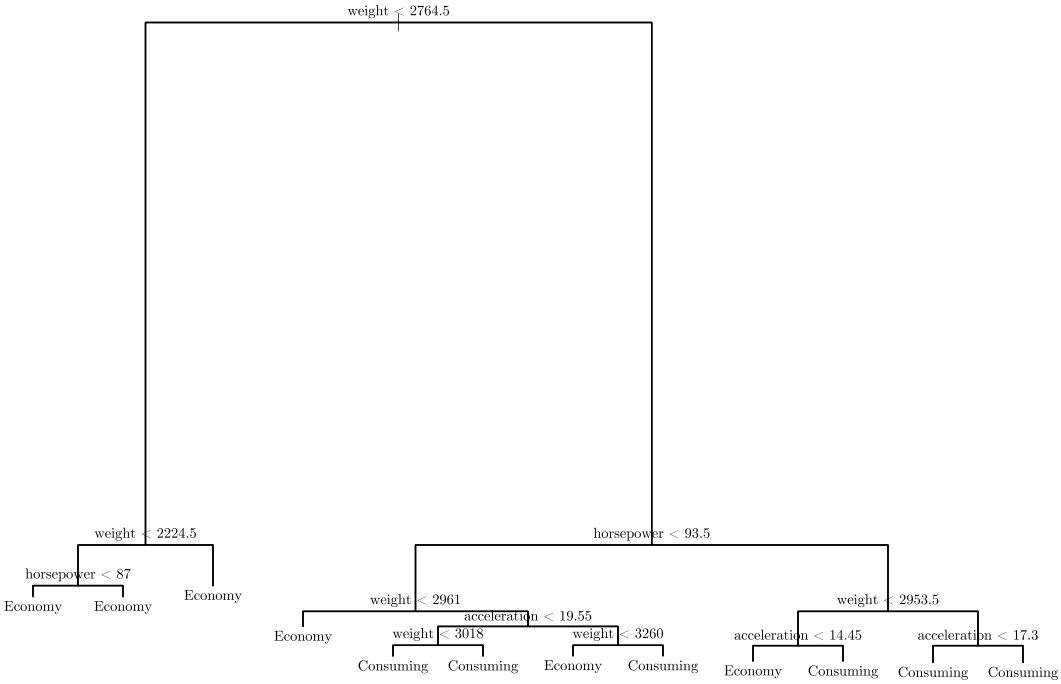

Figure 1: Classification tree for ECO using horsepower, weight, and acceleration.

Once `mpg` is removed, the tree becomes much larger and needs all three technical variables to separate fuel-efficient from fuel-consuming cars. The model now has 12 terminal nodes and a training misclassification error of $29/392 = 7.40\%$, so the problem is no longer trivial.

Problem 1 (6)

Problem 1 (6)

What is the most important variable? Explain the first internal node for me.

The most important variable is `weight`. It appears at the root split, and it is also reused several times deeper in the tree, so it is doing most of the structural work. The first internal node splits at `weight < 2764.5`. Among the 191 cars below 2764.5 pounds, 90.05% are classified as `Economy`, whereas among the 201 cars above 2764.5 pounds, 88.06% are classified as `Consuming`. So the first split says that lighter cars tend to be fuel-efficient, while heavier cars tend to consume more fuel.

Problem 1 (7)

Problem 1 (7)

Split the data into training and test data using

$$n = \texttt{length(Auto\$ECO)}, \quad \texttt{set.seed(1)}, \quad \texttt{Z = sample(n, n/2)},$$

fit tree on `Auto[Z, ]` using `horsepower + weight + acceleration`, and report classification error rate on test data.

Cell 8

```r
# Split Auto into training, test sets.
set.seed(1)
n_auto <- length(auto_df$ECO)
Z_auto <- sample(n_auto, n_auto / 2)
auto_train <- auto_df[Z_auto, ]
auto_test  <- auto_df[-Z_auto, ]

# Prepare table.
auto_split_tbl <- tibble::tibble(
  set = c('Training', 'Test'),
  observations = c(nrow(auto_train), nrow(auto_test))
)

# Print split table.
auto_split_tbl %>%
  table_latex(
    col_names = c('Set', 'Observations'),
    caption   = 'Training-test split for Auto.'
  )
```

Table 2: Training-test split for Auto.

Set	Observations
Training	196
Test	196

Cell 9

```r
# Fit training tree, predict on test set.
auto_tree_train <- tree::tree(ECO ~ horsepower + weight + acceleration, data =
auto_train)
auto_tree_train
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##  1) root 196 271.400 Economy ( 0.47959 0.52041 )
##    2) weight < 2956 113  92.170 Economy ( 0.14159 0.85841 )
```

```
##      4) horsepower < 85.5 64  10.300 Economy ( 0.01562 0.98438 )
##        8) weight < 2481 54   0.000 Economy ( 0.00000 1.00000 ) *
##        9) weight > 2481 10   6.502 Economy ( 0.10000 0.90000 ) *
##      5) horsepower > 85.5 49  60.360 Economy ( 0.30612 0.69388 )
##       10) weight < 2759.5 37  38.630 Economy ( 0.21622 0.78378 ) *
##       11) weight > 2759.5 12  16.300 Consuming ( 0.58333 0.41667 )
##         22) weight < 2850 6   5.407 Consuming ( 0.83333 0.16667 ) *
##         23) weight > 2850 6   7.638 Economy ( 0.33333 0.66667 ) *
##    3) weight > 2956 83  37.790 Consuming ( 0.93976 0.06024 )
##      6) acceleration < 19.8 78  10.700 Consuming ( 0.98718 0.01282 )
##       12) horsepower < 87.5 5   5.004 Consuming ( 0.80000 0.20000 ) *
##       13) horsepower > 87.5 73   0.000 Consuming ( 1.00000 0.00000 ) *
##      7) acceleration > 19.8 5   5.004 Economy ( 0.20000 0.80000 ) *
```

Cell 10

```r
auto_test_pred  <- predict(auto_tree_train, newdata = auto_test, type = 'class')
auto_test_error <- class_error(auto_test$ECO, auto_test_pred)
```

Cell 11

```r
# Print confusion matrix for test data.
confusion_df(auto_test_pred, auto_test$ECO) %>%
  table_latex(
    col_names = c('Predicted', 'Truth = Consuming', 'Truth = Economy'),
    caption   = 'Test-set confusion matrix for the unpruned Auto tree.'
  )
```

Table 3: Test-set confusion matrix for the unpruned Auto tree.

Predicted	Truth = Consuming	Truth = Economy
Consuming	83	6
Economy	19	88

Using the required split, I obtain a test misclassification error of

$$\boxed{\frac{25}{196} = 0.1276}.$$

The tree correctly classifies 83 of the 102 `Consuming` cars and 88 of the 94 `Economy` cars in the test set.

Problem 1 (8)

Problem 1 (8)

Prune tree using `cv.tree()` with misclassification rate as criterion. What number of terminal nodes would you decide on, and why?

Cell 12

```r
# Cross-validation for pruning with misclassification criterion.
auto_cv <- tree::cv.tree(auto_tree_train, FUN = prune.misclass)

# Prepare table.
auto_cv_tbl <- tibble::tibble(
  tree_size     = auto_cv$size,
  cv_misclass   = auto_cv$dev,
  cv_error_rate = round(auto_cv$dev / nrow(auto_train), 4)
)
auto_cv_tbl <- auto_cv_tbl[order(auto_cv_tbl$tree_size), ]

# Print cross-validation table.
auto_cv_tbl %>%
  table_latex(
    col_names = c('Terminal Nodes', 'CV Misclassifications', 'CV Error Rate'),
    caption   = 'Cross-validation results for pruning the Auto classification tree.'
  )
```

Table 4: Cross-validation results for pruning the Auto classification tree.

Terminal Nodes	CV Misclassifications	CV Error Rate
1	102	0.5204
2	28	0.1429
3	28	0.1429
6	20	0.1020
8	21	0.1071

Cell 13

```r
# Plot cross-validated classification error rate.
with_family({
  plot(
    auto_cv_tbl$tree_size,
    auto_cv_tbl$cv_error_rate,
    type = 'b',
    xlab = 'Tree Size',
    ylab = 'CV Classification Error Rate'
  )
})
```

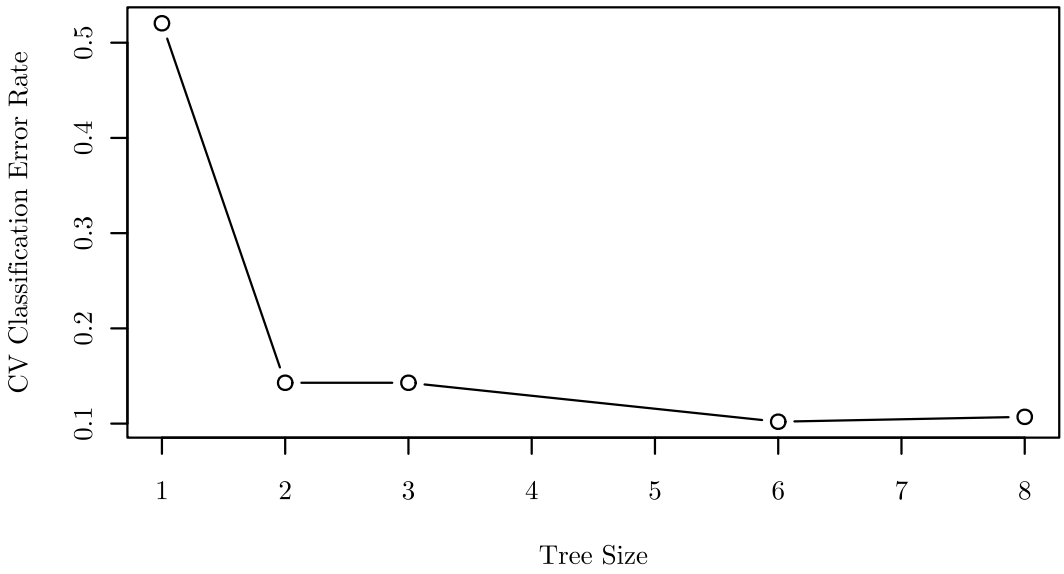

Figure 2: Cross-validated classification error rate versus tree size for Auto.

I would choose the tree with $\boxed{2}$ terminal nodes. From the graph, the largest improvement happens immediately when we move away from the 1-node tree, but after that the cross-validated error decreases only modestly as we continue adding more terminal nodes. So the broad picture is that most of the meaningful reduction in CV error is captured very early, while the larger trees mainly add complexity for comparatively small additional gains. For that reason, the 2-node tree is the more parsimonious choice.

Problem 1 (9)

Problem 1 (9)

Prune the tree to the optimal size, show the tree, and report the new classification error rate.

Cell 14

```r
# Prune Auto tree to selected size.
auto_selected_size <- 2
auto_pruned <- tree::prune.misclass(auto_tree_train, best = auto_selected_size)
auto_pruned
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
## 1) root 196 271.40 Economy ( 0.47959 0.52041 )
##   2) weight < 2956 113  92.17 Economy ( 0.14159 0.85841 ) *
##   3) weight > 2956 83  37.79 Consuming ( 0.93976 0.06024 ) *
```

Cell 15

```r
# Plot pruned tree with text.
with_family({
  plot(auto_pruned)
  text(auto_pruned, pretty = 0, cex = 0.8)
})
```

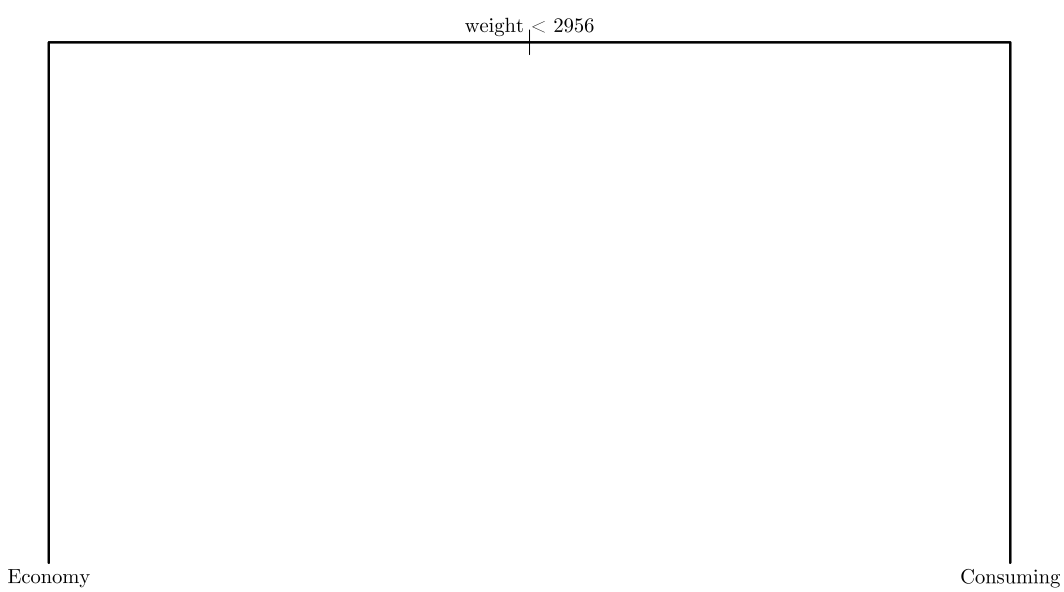

Figure 3: Pruned Auto classification tree with 2 terminal nodes.

Cell 16

```r
# Compute pruned-tree test confusion matrix.
```

```r
auto_pruned_pred <- predict(auto_pruned, newdata = auto_test, type = 'class')
auto_pruned_test_error <- class_error(auto_test$ECO, auto_pruned_pred)
confusion_df(auto_pruned_pred, auto_test$ECO) %>%
  table_latex(
    col_names = c('Predicted', 'Truth = Consuming', 'Truth = Economy'),
    caption   = 'Test-set confusion matrix for the pruned Auto tree.'
  )
```

Table 5: Test-set confusion matrix for the pruned Auto tree.

Predicted	Truth = Consuming	Truth = Economy
Consuming	83	7
Economy	19	87

After pruning to 2 terminal nodes, the test misclassification error is

$$\boxed{\frac{26}{196} = 0.1327}.$$

So the simpler 2-node tree has slightly higher test error than the original 8-node tree, but the increase is small.

Problem 1 (10)

Problem 1 (10)

Compare the tree before pruning and after pruning. What are the differences?

Cell 17

```r
# Compare unpruned and pruned Auto trees.
auto_compare_tbl <- tibble::tibble(
  tree = c('Unpruned', 'Pruned'),
  terminal_nodes = c(summary(auto_tree_train)$size, summary(auto_pruned)$size),
  training_error = c(
    round(class_error(auto_train$ECO, predict(auto_tree_train, auto_train, type =
    'class')), 4),
    round(class_error(auto_train$ECO, predict(auto_pruned, auto_train, type =
    'class')), 4)
  ),
  test_error = c(round(auto_test_error, 4), round(auto_pruned_test_error, 4))
)

# Print comparison table.
auto_compare_tbl %>%
  table_latex(
    col_names = c('Tree', 'Terminal Nodes', 'Training Error', 'Test Error'),
    caption   = 'Comparison of the Auto tree before and after pruning.'
  )
```

Table 6: Comparison of the Auto tree before and after pruning.

Tree	Terminal Nodes	Training Error	Test Error
Unpruned	8	0.0714	0.1276
Pruned	2	0.1071	0.1327

The main difference is complexity: the original tree has 8 terminal nodes, while the pruned tree has only 2. So pruning now removes most of the smaller follow-up splits and keeps only the broadest separation in the data. This increases the training error from 0.0714 to 0.1071 and increases the test error slightly from 0.1276 to 0.1327. In exchange, the tree becomes much simpler and less likely to reflect small sample-specific splits.

Problem 1 (11a)

Problem 1 (11a)

Treat `Sales` as quantitative response, split data into training set and test set.

Cell 18

```r
# Load Carseats, split into training, test sets.
carseats_df <- ISLR2::Carseats
set.seed(1)
n_carseats <- nrow(carseats_df)
Z_carseats <- sample(n_carseats, n_carseats / 2)
carseats_train <- carseats_df[Z_carseats, ]
carseats_test  <- carseats_df[-Z_carseats, ]

# Prepare table.
carseats_split_tbl <- tibble::tibble(
  set = c('Training', 'Test'),
  observations = c(nrow(carseats_train), nrow(carseats_test))
)

# Print split table.
carseats_split_tbl %>%
  table_latex(
    col_names = c('Set', 'Observations'),
    caption   = 'Training-test split for Carseats.'
  )
```

Table 7: Training-test split for Carseats.

Set	Observations
Training	200
Test	200

Since `Carseats` has 400 observations, this split gives 200 training observations and 200 test observations.

Problem 1 (11b)

Problem 1 (11b)

Fit regression tree to training set. Plot tree, interpret results, and report test MSE.

Cell 19

```r
# Fit regression tree on Carseats training data.
carseats_tree <- tree::tree(Sales ~ ., data = carseats_train)
carseats_tree
```

```
## node), split, n, deviance, yval
##       * denotes terminal node
##
##   1) root 200 1573.000  7.578
##     2) ShelveLoc: Bad,Medium 158  964.600  6.908
##       4) Price < 94.5 24  110.900  9.285
##         8) Age < 39.5 6    9.117 11.640 *
##         9) Age > 39.5 18   57.310  8.499
##          18) Price < 71 6    7.686 10.260 *
##          19) Price > 71 12   21.610  7.617 *
##       5) Price > 94.5 134  694.000  6.483
##        10) Advertising < 4 59  229.900  5.511
##          20) Price < 130 37  135.300  6.136
##            40) CompPrice < 123.5 16   45.860  4.905
##              80) ShelveLoc: Bad 7    8.892  3.487 *
##              81) ShelveLoc: Medium 9   11.950  6.008 *
##            41) CompPrice > 123.5 21   46.760  7.074 *
##          21) Price > 130 22   55.810  4.459
##            42) CompPrice < 137.5 15   28.510  3.859 *
##            43) CompPrice > 137.5 7   10.300  5.746 *
##        11) Advertising > 4 75  364.400  7.247
##          22) Age < 58.5 43  138.700  8.117
##            44) Price < 127 25   55.660  9.055
##              88) CompPrice < 129.5 13   18.830  8.184 *
##              89) CompPrice > 129.5 12   16.260  9.999 *
##            45) Price > 127 18   30.440  6.814 *
##          23) Age > 58.5 32  149.500  6.079
##            46) ShelveLoc: Bad 6   17.310  4.003 *
##            47) ShelveLoc: Medium 26  100.400  6.558
##              94) CompPrice < 123.5 13   24.930  5.477 *
##              95) CompPrice > 123.5 13   45.120  7.638
##               190) Price < 124.5 6    4.535  9.153 *
##               191) Price > 124.5 7   15.010  6.340 *
##     3) ShelveLoc: Good 42  270.500 10.100
##       6) Price < 135 30  116.900 11.150
##        12) US: No 8   37.410  9.451 *
##        13) US: Yes 22   47.880 11.770 *
##       7) Price > 135 12   36.890  7.462 *
```

Cell 20

```r
summary(carseats_tree)
```

```
##
## Regression tree:
## tree::tree(formula = Sales ~ ., data = carseats_train)
## Variables actually used in tree construction:
## [1] "ShelveLoc"   "Price"       "Age"         "Advertising" "CompPrice"   "US"
```

```
## Number of terminal nodes:  18
## Residual mean deviance:  2.167 = 394.3 / 182
## Distribution of residuals:
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -3.88200 -0.88200 -0.08712  0.00000  0.89590  4.09900
```

Cell 21

```r
carseats_pred <- predict(carseats_tree, newdata = carseats_test)
carseats_mse  <- test_mse(carseats_test$Sales, carseats_pred)
```

Cell 22

```r
# Plot Carseats regression tree with text.
with_family({
  plot(carseats_tree)
  text(carseats_tree, pretty = 0, cex = 0.6)
})
```

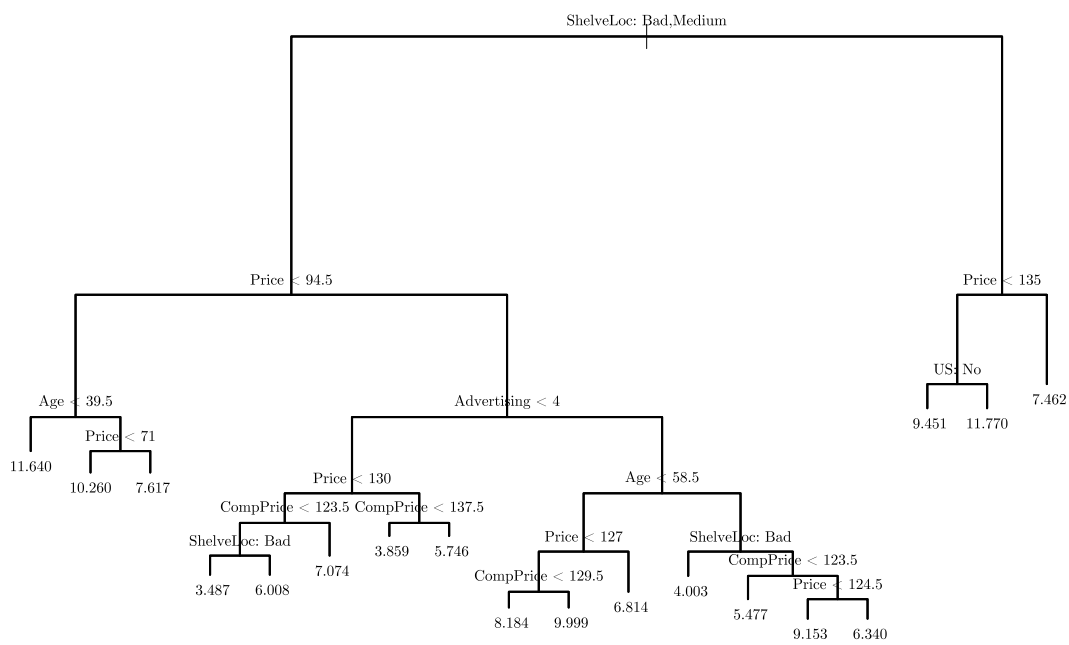

Figure 4: Regression tree for Sales on the Carseats training set.

Cell 23

```r
# Summarize fitted regression tree.
carseats_tree_tbl <- tibble::tibble(
  terminal_nodes = summary(carseats_tree)$size,
  test_mse = round(carseats_mse, 4)
)

# Print tree summary table.
carseats_tree_tbl %>%
  table_latex(
```

```
    col_names = c('Terminal Nodes', 'Test MSE'),
    caption   = 'Regression-tree performance on Carseats.'
  )
```

Table 8: Regression-tree performance on Carseats.

Terminal Nodes	Test MSE
18	4.922

The regression tree first splits on `ShelveLoc`, which already tells us that shelf quality is the most important driver of sales in this tree. Stores with `ShelveLoc = Good` tend to have much higher predicted sales than stores with `ShelveLoc = Bad` or `Medium`. Inside those branches, `Price`, `Advertising`, `CompPrice`, `Age`, and `US` further refine the predictions. The fitted tree has 18 terminal nodes, and its test MSE is $\boxed{4.9220}$.

Problem 1 (11c)

Problem 1 (11c)

Use cross-validation to determine optimal level of tree complexity. Does pruning tree improve test MSE?

Cell 24

```r
# Cross-validation for regression-tree pruning.
carseats_cv <- tree::cv.tree(carseats_tree)

# Prepare table.
carseats_cv_tbl <- tibble::tibble(
  tree_size = carseats_cv$size,
  cv_deviance = round(carseats_cv$dev, 4)
)
carseats_cv_tbl <- carseats_cv_tbl[order(carseats_cv_tbl$tree_size), ]

# Print cross-validation table.
carseats_cv_tbl %>%
  table_latex(
    col_names = c('Terminal Nodes', 'CV Deviance'),
    caption   = 'Cross-validation results for pruning Carseats regression tree.'
  )
```

Table 9: Cross-validation results for pruning Carseats regression tree.

Terminal Nodes	CV Deviance
1	1620.4687
2	1349.9273
3	1302.8607
4	1273.2889
5	1205.5806
6	1125.0894
7	1066.6899
8	1039.1241
10	955.2535
11	950.2691
12	911.2580
13	893.4641
14	862.3400
15	862.3400
16	868.6815
17	852.3639
18	831.3437

Cell 25

```r
# Plot cross-validation deviance against tree size.
with_family({
  plot(
    carseats_cv_tbl$tree_size,
    carseats_cv_tbl$cv_deviance,
    type = 'b',
    xlab = 'Tree Size',
    ylab = 'CV Deviance'
  )
})
```

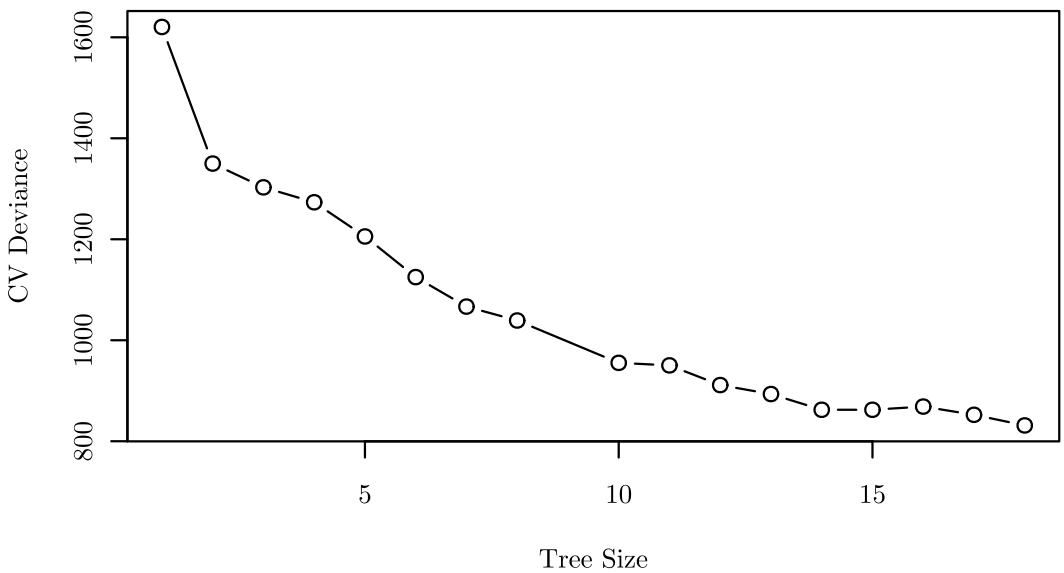

Figure 5: Cross-validation deviance versus tree size for the Carseats regression tree.

Cell 26

```r
# Prune to selected size, compare test MSE.
carseats_selected_size <- 10
carseats_pruned <- tree::prune.tree(carseats_tree, best = carseats_selected_size)
carseats_pruned_pred <- predict(carseats_pruned, newdata = carseats_test)
carseats_pruned_mse <- test_mse(carseats_test$Sales, carseats_pruned_pred)

# Prepare table.
carseats_prune_tbl <- tibble::tibble(
  tree = c('Unpruned', 'Pruned'),
  terminal_nodes = c(summary(carseats_tree)$size, summary(carseats_pruned)$size),
  test_mse = round(c(carseats_mse, carseats_pruned_mse), 4)
)

# Print pruning comparison.
carseats_prune_tbl %>%
```

```r
table_latex(
  col_names = c('Tree', 'Terminal Nodes', 'Test MSE'),
  caption   = 'Carseats test MSE before and after pruning.'
)
```

Table 10: Carseats test MSE before and after pruning.

Tree	Terminal Nodes	Test MSE
Unpruned	18	4.9220
Pruned	10	4.9181

I would choose $\boxed{10}$ terminal nodes rather than the full 18-node tree. From the cross-validation plot, the deviance drops substantially as we move away from the very smallest trees, but once we get to about size 10 the additional decreases from larger trees are relatively modest compared with the extra complexity. So by 10 terminal nodes, most of the important reduction in cross-validated error has already been achieved, and the gains from continuing to larger trees are comparatively small. With this pruned tree, the test MSE is $\boxed{4.9181}$, compared with 4.9220 for the unpruned tree. So pruning does not materially change performance here, but the 10-node tree is slightly better on the test set while also being substantially simpler.

Problem 1 (11d)

Problem 1 (11d)

Use bagging to analyze this data. What test MSE do you obtain? Use `importance()` to determine which variables are most important.

Cell 27

```r
# Fit bagging model on Carseats.
set.seed(1)
carseats_bag <- randomForest::randomForest(
  Sales ~ .,
  data = carseats_train,
  mtry = ncol(carseats_train) - 1,
  importance = TRUE
)

# Prepare tables.
carseats_bag_pred <- predict(carseats_bag, newdata = carseats_test)
carseats_bag_mse  <- test_mse(carseats_test$Sales, carseats_bag_pred)
carseats_imp      <- randomForest::importance(carseats_bag)
carseats_imp_tbl <- data.frame(
  variable        = rownames(carseats_imp),
  pct_inc_mse     = round(carseats_imp[, '%IncMSE'], 3),
  inc_node_purity = round(carseats_imp[, 'IncNodePurity'], 3),
  row.names       = NULL
)
carseats_imp_tbl <- carseats_imp_tbl[order(-carseats_imp_tbl$pct_inc_mse), ]
carseats_bag_tbl <- tibble::tibble(
  method = 'Bagging',
  mtry = ncol(carseats_train) - 1,
  test_mse = round(carseats_bag_mse, 4)
)

# Print bagging performance.
carseats_bag_tbl %>%
  table_latex(
    col_names = c('Method', '$m$', 'Test MSE'),
    caption   = 'Bagging performance on Carseats.',
    escape    = FALSE
  )
```

Table 11: Bagging performance on Carseats.

Method	m	Test MSE
Bagging	10	2.6053

Cell 28

```r
# Print variable importance.
carseats_imp_tbl %>%
  table_latex(
    col_names = c('Variable', '\\%IncMSE', 'IncNodePurity'),
    caption   = 'Variable importance from the bagging model on Carseats.',
    escape    = FALSE
  )
```

Table 12: Variable importance from the bagging model on Carseats.

	Variable	%IncMSE	IncNodePurity
5	Price	56.333	502.903
6	ShelveLoc	48.889	380.033
1	CompPrice	24.889	170.183
7	Age	17.728	157.847
3	Advertising	12.769	97.164
2	Income	4.712	91.265
10	US	4.217	18.074
8	Education	0.596	44.599
9	Urban	0.173	9.822
4	Population	-1.807	58.245

The bagging model reduces the test MSE to $\boxed{2.6053}$, which is much lower than regression-tree test MSE of 4.9220. Here I used `mtry = ncol(carseats_train) - 1 = 10`, so every predictor is available at every split because bagging is a special case of random forests with $m = p$. According to `importance()`, the two dominant variables are `Price` and `ShelveLoc`. After those, the next most important variables are `CompPrice`, `Age`, and `Advertising`. So ensemble substantially improves predictive accuracy, and it confirms that product price and shelf quality are main drivers of sales in this data.

Problem 2

Problem 2

This question relates to the plots in Figure 8.14. I reproduce the requested tree sketch and predictor-space partition directly from the figure.

Problem 2 (a)

Problem 2 (a)

Sketch the tree corresponding to the partition of the predictor space illustrated in the left-hand panel of Figure 8.14.

Cell 29

```r
# Draw tree corresponding to left-hand partition in Figure 8.14.
with_family({
  plot.new()
  plot.window(xlim = c(0, 1), ylim = c(0, 1))
  segments(0.50, 0.92, 0.50, 0.82)
  text(0.50, 0.95, 'X1 < 1')

  segments(0.50, 0.82, 0.30, 0.68)
  segments(0.50, 0.82, 0.78, 0.68)
  text(0.78, 0.66, '5')

  segments(0.30, 0.68, 0.30, 0.60)
  text(0.30, 0.72, 'X2 < 1')

  segments(0.30, 0.60, 0.18, 0.46)
  segments(0.30, 0.60, 0.42, 0.46)
  text(0.42, 0.44, '15')

  segments(0.18, 0.46, 0.18, 0.38)
  text(0.18, 0.50, 'X1 < 0')

  segments(0.18, 0.38, 0.10, 0.24)
  segments(0.18, 0.38, 0.26, 0.24)
  text(0.10, 0.22, '3')

  segments(0.26, 0.24, 0.26, 0.16)
  text(0.26, 0.28, 'X2 < 0')

  segments(0.26, 0.16, 0.20, 0.04)
  segments(0.26, 0.16, 0.32, 0.04)
  text(0.20, 0.02, '10')
  text(0.32, 0.02, '0')
})
```

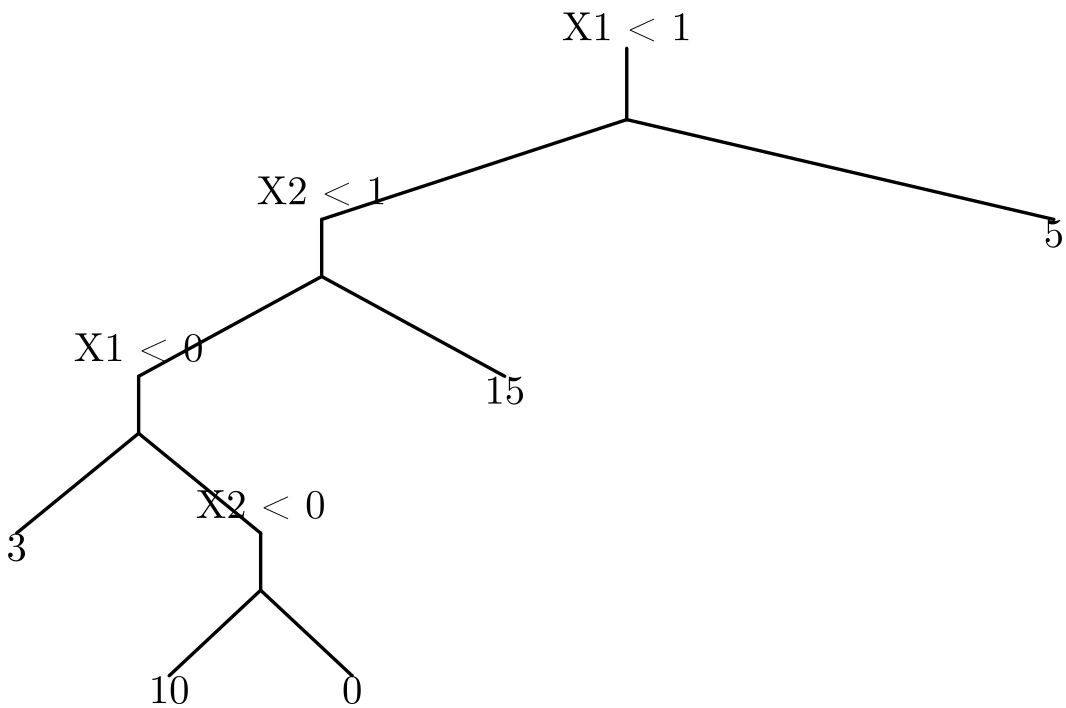

Figure 6: Tree corresponding to the left-hand partition in Figure 8.14.

The corresponding tree starts with a split at $X_1 = 1$, since the left panel first divides the whole predictor space vertically there. Using the usual convention that the left branch is the `yes` branch for the condition shown at the node, the right child of the root is the region with mean 5, because that branch corresponds to $X_1 \geq 1$. Inside the left region $X_1 < 1$, the next split is $X_2 < 1$: the left child is the lower part of the rectangle, while the right child is the upper rectangle with mean 15. The lower-left region then splits at $X_1 = 0$, sending the region with mean 3 to the left and the middle strip to the right, and that final strip splits at $X_2 = 0$ to produce terminal-node means 10 and 0. So the terminal-node means are 3, 10, 0, 15, and 5, exactly matching the partition.

Problem 2 (b)

Problem 2 (b)

Create a diagram similar to the left-hand panel of Figure 8.14, using the tree illustrated in the right-hand panel of the same figure.

Cell 30

```r
# Draw partition implied by right-hand tree in Figure 8.14.
with_family({
  plot(
    0, 0,
    type = 'n',
    xlim = c(-2, 2.5),
    ylim = c(0, 3),
    xlab = expression(X[1]),
    ylab = expression(X[2]),
    axes = FALSE
  )
  axis(1)
  axis(2)
  box()

  abline(h = 1, lwd = 1.2)
  abline(h = 2, lwd = 1.2)
  segments(1, 0, 1, 1, lwd = 1.2)
  segments(0, 1, 0, 2, lwd = 1.2)

  text(-0.5, 0.5, '-1.80')
  text(1.75, 0.5, '0.63')
  text(-1.0, 1.5, '-1.06')
  text(1.0, 1.5, '0.21')
  text(0.25, 2.5, '2.49')
})
```

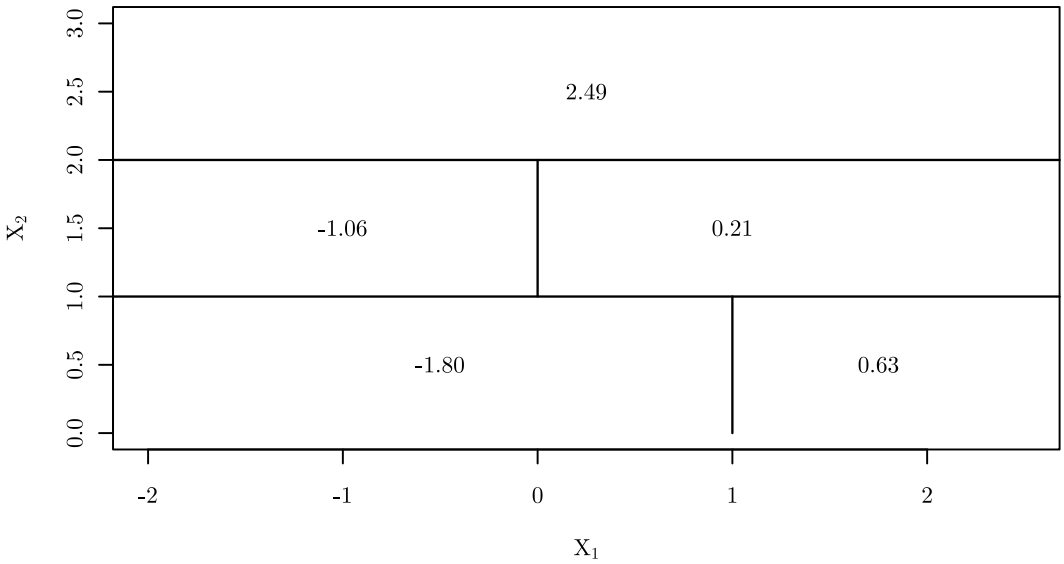

Figure 7: Predictor-space partition implied by the right-hand tree in Figure 8.14.

Reading the right-hand tree from top to bottom, the first split is $X_2 < 1$, so the predictor space is first cut horizontally at $X_2 = 1$. For the region with $X_2 \geq 1$, the next split is $X_2 < 2$, so we add a second horizontal cut at $X_2 = 2$. Within the middle band $1 \leq X_2 < 2$, the tree then splits at $X_1 = 0$. Within the bottom band $X_2 < 1$, the tree splits at $X_1 = 1$. The five terminal-region means are therefore -1.80, 0.63, -1.06, 0.21, and 2.49.

Problem 3

Problem 3

This problem uses `OJ` data from `ISLR2` and classification trees for predicting `Purchase`.

Problem 3 (a)

Problem 3 (a)

Create a training set containing a random sample of 800 observations and a test set containing the remaining observations.

Cell 31

```r
# Load OJ, split into training, test sets.
oj_df <- ISLR2::OJ
set.seed(1)
oj_train_idx <- sample(seq_len(nrow(oj_df)), 800)
oj_train <- oj_df[oj_train_idx, ]
oj_test  <- oj_df[-oj_train_idx, ]

# Prepare table.
oj_split_tbl <- tibble::tibble(
  set = c('Training', 'Test'),
  observations = c(nrow(oj_train), nrow(oj_test))
)

# Print split table.
oj_split_tbl %>%
  table_latex(
    col_names = c('Set', 'Observations'),
    caption   = 'Training-test split for OJ.'
  )
```

Table 13: Training-test split for OJ.

Set	Observations
Training	800
Test	270

This gives a training set of 800 observations and a test set of 270 observations.

Problem 3 (b)

Problem 3 (b)

Fit a tree to training data, with `Purchase` as response and other variables as predictors. Use `summary()` to describe results. What is training error rate? How many terminal nodes does tree have?

Cell 32

```r
# Fit classification tree on OJ training data.
oj_tree <- tree::tree(Purchase ~ ., data = oj_train)
summary(oj_tree)
```

```
##
## Classification tree:
## tree::tree(formula = Purchase ~ ., data = oj_train)
## Variables actually used in tree construction:
## [1] "LoyalCH"       "PriceDiff"     "SpecialCH"     "ListPriceDiff" "PctDiscMM"
## Number of terminal nodes:  9
## Residual mean deviance:  0.7432 = 587.8 / 791
## Misclassification error rate: 0.1588 = 127 / 800
```

Cell 33

```r
# Summarize training performance.
oj_train_pred  <- predict(oj_tree, newdata = oj_train, type = 'class')
oj_train_error <- class_error(oj_train$Purchase, oj_train_pred)

# Prepare table.
oj_summary_tbl <- tibble::tibble(
  terminal_nodes = summary(oj_tree)$size,
  training_error = round(oj_train_error, 4)
)

# Print OJ summary table.
oj_summary_tbl %>%
  table_latex(
    col_names = c('Terminal Nodes', 'Training Error'),
    caption   = 'Summary of the fitted OJ classification tree.'
  )
```

Table 14: Summary of the fitted OJ classification tree.

Terminal Nodes	Training Error
9	0.1588

The tree uses only five predictors: `LoyalCH`, `PriceDiff`, `SpecialCH`, `ListPriceDiff`, and `PctDiscMM`. The most important by far is `LoyalCH`, since it appears in the first split and again in the upper part of the tree. The fitted tree has $\boxed{9}$ terminal nodes, and the training

misclassification error is

$$\boxed{\frac{127}{800} = 0.1588}.$$

Problem 3 (c)

Problem 3 (c)

Type in name of tree object in order to get detailed text output. Pick one terminal node, and interpret information displayed.

Cell 34

```
# Detailed text output for OJ tree.
oj_tree
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##  1) root 800 1073.00 CH ( 0.60625 0.39375 )
##    2) LoyalCH < 0.5036 365  441.60 MM ( 0.29315 0.70685 )
##      4) LoyalCH < 0.280875 177  140.50 MM ( 0.13559 0.86441 )
##        8) LoyalCH < 0.0356415 59   10.14 MM ( 0.01695 0.98305 ) *
##        9) LoyalCH > 0.0356415 118  116.40 MM ( 0.19492 0.80508 ) *
##      5) LoyalCH > 0.280875 188  258.00 MM ( 0.44149 0.55851 )
##       10) PriceDiff < 0.05 79   84.79 MM ( 0.22785 0.77215 )
##         20) SpecialCH < 0.5 64   51.98 MM ( 0.14062 0.85938 ) *
##         21) SpecialCH > 0.5 15   20.19 CH ( 0.60000 0.40000 ) *
##       11) PriceDiff > 0.05 109  147.00 CH ( 0.59633 0.40367 ) *
##    3) LoyalCH > 0.5036 435  337.90 CH ( 0.86897 0.13103 )
##      6) LoyalCH < 0.764572 174  201.00 CH ( 0.73563 0.26437 )
##       12) ListPriceDiff < 0.235 72   99.81 MM ( 0.50000 0.50000 )
##         24) PctDiscMM < 0.196196 55   73.14 CH ( 0.61818 0.38182 ) *
##         25) PctDiscMM > 0.196196 17   12.32 MM ( 0.11765 0.88235 ) *
##       13) ListPriceDiff > 0.235 102   65.43 CH ( 0.90196 0.09804 ) *
##      7) LoyalCH > 0.764572 261   91.20 CH ( 0.95785 0.04215 ) *
```

Consider terminal node 8. It corresponds to customers with `LoyalCH < 0.0356415`. That node contains 59 observations and predicts `MM`, with estimated class probabilities $(0.01695, 0.98305)$ for (`CH`, `MM`). So for customers with extremely low loyalty to Citrus Hill, the tree says that a Minute Maid purchase is overwhelmingly likely.

Problem 3 (d)

Problem 3 (d)

Create a plot of the tree, and interpret the results.

Cell 35

```r
# Plot OJ classification tree.
with_family({
  plot(oj_tree)
  text(oj_tree, pretty = 0, cex = 0.70)
})
```

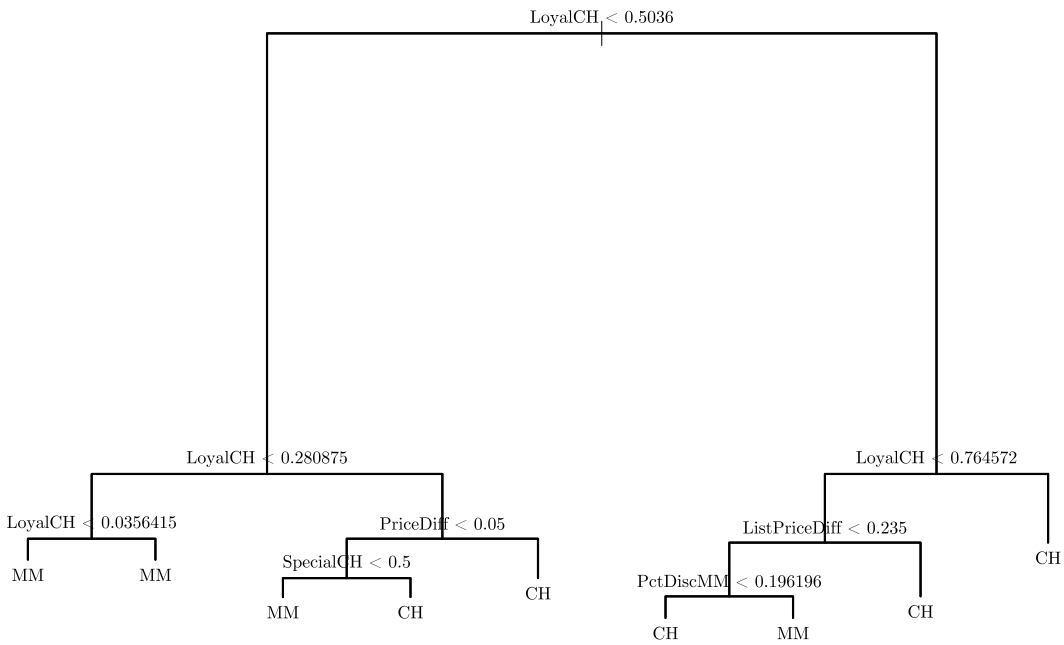

Figure 8: Classification tree for Purchase on the OJ training set.

The plot reinforces the same message as the text output: `LoyalCH` dominates the tree. Customers with low loyalty to Citrus Hill are mostly classified as `MM`, while customers with high loyalty to Citrus Hill are mostly classified as `CH`. Price-related variables such as `PriceDiff` and `ListPriceDiff`, along with promotions like `SpecialCH` and `PctDiscMM`, only come in after loyalty has already partitioned the data into broad groups.

Problem 3 (e)

Problem 3 (e)

Predict the response on the test data, produce a confusion matrix, and report the test error rate.

Cell 36

```r
# Predict Purchase on OJ test set.
oj_test_pred  <- predict(oj_tree, newdata = oj_test, type = 'class')
oj_test_error <- class_error(oj_test$Purchase, oj_test_pred)
confusion_df(oj_test_pred, oj_test$Purchase) %>%
  table_latex(
    col_names = c('Predicted', 'Truth = CH', 'Truth = MM'),
    caption   = 'Test-set confusion matrix for the unpruned OJ tree.'
  )
```

Table 15: Test-set confusion matrix for the unpruned OJ tree.

Predicted	Truth = CH	Truth = MM
CH	160	38
MM	8	64

The test misclassification error is

$$\boxed{\frac{46}{270} = 0.1704}.$$

The tree correctly classifies 160 of the 168 `CH` observations and 64 of the 102 `MM` observations in the test set.

Problem 3 (f)

Problem 3 (f)

Apply `cv.tree()` to training set in order to determine optimal tree size.

Cell 37

```r
# Cross-validation for pruning OJ tree.
oj_cv <- tree::cv.tree(oj_tree, FUN = prune.misclass)

# Prepare table.
oj_cv_tbl <- tibble::tibble(
  tree_size     = oj_cv$size,
  cv_misclass   = oj_cv$dev,
  cv_error_rate = round(oj_cv$dev / nrow(oj_train), 4)
)
oj_cv_tbl <- oj_cv_tbl[order(oj_cv_tbl$tree_size), ]

# Print cross-validation table.
oj_cv_tbl %>%
  table_latex(
    col_names = c('Terminal Nodes', 'CV Misclassifications', 'CV Error Rate'),
    caption   = 'Cross-validation results for pruning OJ classification tree.'
  )
```

Table 16: Cross-validation results for pruning OJ classification tree.

Terminal Nodes	CV Misclassifications	CV Error Rate
1	315	0.3938
2	172	0.2150
4	158	0.1975
7	149	0.1862
8	150	0.1875
9	150	0.1875

This cross-validation output gives candidate tree sizes and corresponding cross-validated misclassification counts. The graph in part (g) and the pruning discussion in part (i) explain how I interpret these results when deciding which tree sizes are the most reasonable choices.

Problem 3 (g)

Problem 3 (g)

Produce a plot with tree size on the x-axis and cross-validated classification error rate on the y-axis.

Cell 38

```r
# Plot cross-validated classification error rate.
with_family({
  plot(
    oj_cv_tbl$tree_size,
    oj_cv_tbl$cv_error_rate,
    type = 'b',
    xlab = 'Tree Size',
    ylab = 'CV Classification Error Rate'
  )
})
```

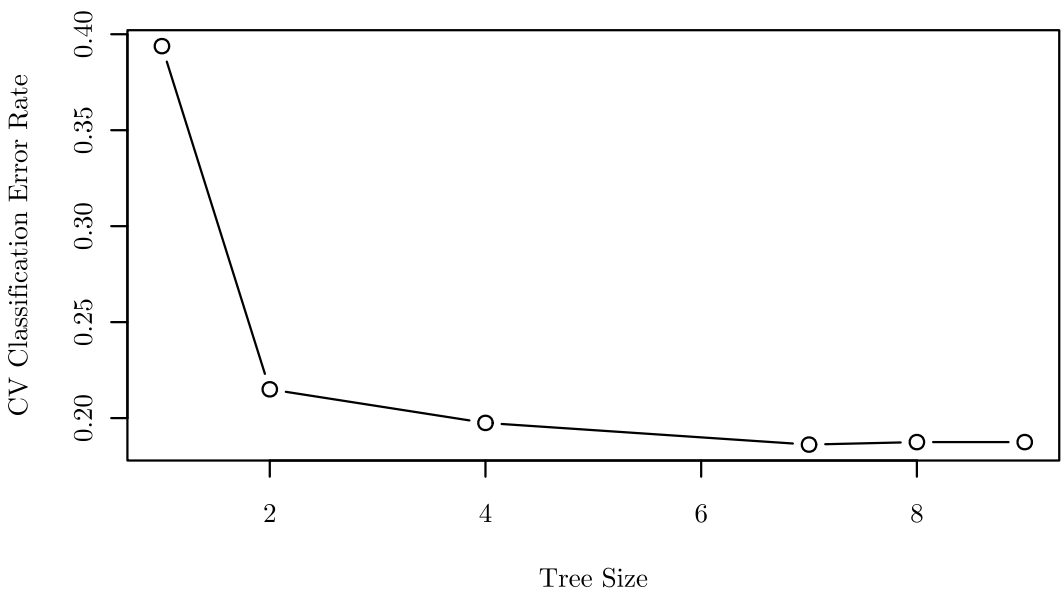

Figure 9: Cross-validated classification error rate versus tree size for the OJ tree.

The curve is lowest at tree size 7, so that is the size that minimizes the cross-validated classification error. At the same time, the plot also shows the broader tradeoff between fit and parsimony: the error drops a lot when we move away from the 1-node tree, but the gains from keeping increasingly larger trees are not uniformly dramatic. So the graph supports two reasonable views of the problem: a 7-node tree if we want to stay at the minimum of the CV curve, and a much smaller 2-node tree if we want to prioritize simplicity more heavily while still capturing a substantial portion of the overall drop in CV error.

Problem 3 (h)

Problem 3 (h)

Which tree size corresponds to the lowest cross-validated classification error rate?

The tree with $\boxed{7}$ terminal nodes has the lowest cross-validated misclassification count, 149, and it is also the smallest tree before the cross-validated error begins to rise more clearly.

Problem 3 (i)

Problem 3 (i)

Produce a pruned tree corresponding to the optimal tree size obtained using cross-validation.

Cell 39

```r
# Prune OJ tree to selected sizes.
oj_pruned_7 <- tree::prune.misclass(oj_tree, best = 7)
oj_pruned_2 <- tree::prune.misclass(oj_tree, best = 2)

oj_pruned_7
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##  1) root 800 1073.00 CH ( 0.60625 0.39375 )
##    2) LoyalCH < 0.5036 365  441.60 MM ( 0.29315 0.70685 )
##      4) LoyalCH < 0.280875 177  140.50 MM ( 0.13559 0.86441 ) *
##      5) LoyalCH > 0.280875 188  258.00 MM ( 0.44149 0.55851 )
##       10) PriceDiff < 0.05 79   84.79 MM ( 0.22785 0.77215 ) *
##       11) PriceDiff > 0.05 109  147.00 CH ( 0.59633 0.40367 ) *
##    3) LoyalCH > 0.5036 435  337.90 CH ( 0.86897 0.13103 )
##      6) LoyalCH < 0.764572 174  201.00 CH ( 0.73563 0.26437 )
##       12) ListPriceDiff < 0.235 72   99.81 MM ( 0.50000 0.50000 )
##         24) PctDiscMM < 0.196196 55   73.14 CH ( 0.61818 0.38182 ) *
##         25) PctDiscMM > 0.196196 17   12.32 MM ( 0.11765 0.88235 ) *
##       13) ListPriceDiff > 0.235 102   65.43 CH ( 0.90196 0.09804 ) *
##      7) LoyalCH > 0.764572 261   91.20 CH ( 0.95785 0.04215 ) *
```

Cell 40

```r
oj_pruned_2
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
## 1) root 800 1073.0 CH ( 0.6062 0.3937 )
##   2) LoyalCH < 0.5036 365  441.6 MM ( 0.2932 0.7068 ) *
##   3) LoyalCH > 0.5036 435  337.9 CH ( 0.8690 0.1310 ) *
```

Cell 41

```r
# Plot 7-node pruned OJ tree.
with_family({
  plot(oj_pruned_7)
  text(oj_pruned_7, pretty = 0, cex = 0.8)
})
```

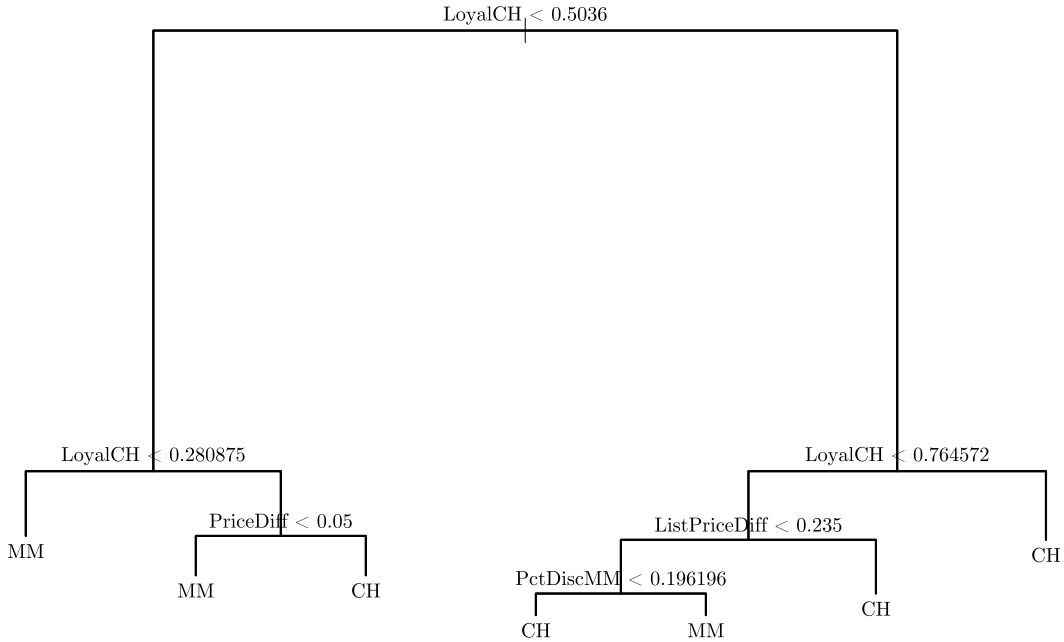

Figure 10: Pruned OJ classification tree with 7 terminal nodes.

Cell 42

```r
# Plot 2-node pruned OJ tree.
with_family({
  plot(oj_pruned_2)
  text(oj_pruned_2, pretty = 0, cex = 0.8)
})
```

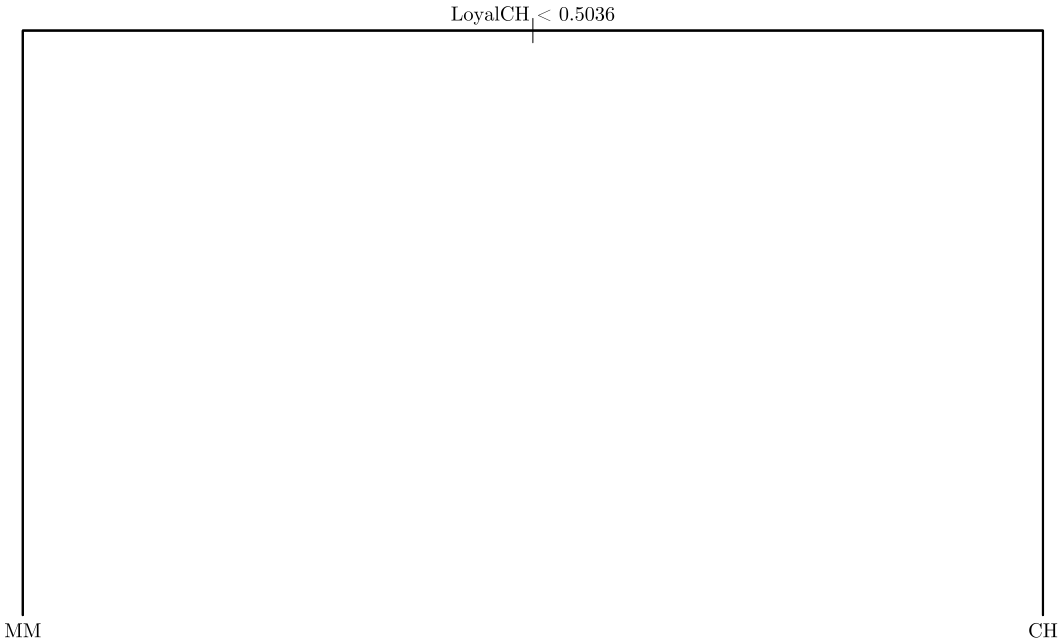

Figure 11: Pruned OJ classification tree with 2 terminal nodes.

Here there are two reasonable pruned choices depending on how strongly we want to prioritize parsimony. The 7-node tree stays at the lowest cross-validated misclassification count, but the 2-node tree is much simpler and already captures a substantial part of the overall reduction in CV error from the 1-node tree. Beyond that, moving to larger trees continues to lower CV error, but the decreases are not dramatic enough to make the choice completely clear if we want to weight simplicity heavily. Since the notion of the "most optimal" tree is therefore somewhat ambiguous, I keep both the 7-node and 2-node pruned trees.

Problem 3 (j)

Problem 3 (j)

Compare the training error rates between the pruned and unpruned trees. Which is higher?

Cell 43

```r
# Compare training, test errors for OJ trees.
oj_pruned_7_train_pred <- predict(oj_pruned_7, newdata = oj_train, type = 'class')
oj_pruned_7_test_pred  <- predict(oj_pruned_7, newdata = oj_test, type = 'class')
oj_pruned_2_train_pred <- predict(oj_pruned_2, newdata = oj_train, type = 'class')
oj_pruned_2_test_pred  <- predict(oj_pruned_2, newdata = oj_test, type = 'class')

oj_pruned_7_train_error <- class_error(oj_train$Purchase, oj_pruned_7_train_pred)
oj_pruned_7_test_error  <- class_error(oj_test$Purchase, oj_pruned_7_test_pred)
oj_pruned_2_train_error <- class_error(oj_train$Purchase, oj_pruned_2_train_pred)
oj_pruned_2_test_error  <- class_error(oj_test$Purchase, oj_pruned_2_test_pred)

# Prepare table.
oj_compare_tbl <- tibble::tibble(
  tree = c('Unpruned', 'Pruned (7 nodes)', 'Pruned (2 nodes)'),
  terminal_nodes = c(summary(oj_tree)$size, summary(oj_pruned_7)$size, summary(oj_pruned_2)$size),
  training_error = round(c(oj_train_error, oj_pruned_7_train_error, oj_pruned_2_train_error), 4),
  test_error = round(c(oj_test_error, oj_pruned_7_test_error, oj_pruned_2_test_error), 4)
)

# Print comparison table.
oj_compare_tbl %>%
  table_latex(
    col_names = c('Tree', 'Terminal Nodes', 'Training Error', 'Test Error'),
    caption   = 'Comparison of the pruned and unpruned OJ trees.'
  )
```

Table 17: Comparison of the pruned and unpruned OJ trees.

Tree	Terminal Nodes	Training Error	Test Error
Unpruned	9	0.1588	0.1704
Pruned (7 nodes)	7	0.1625	0.1630
Pruned (2 nodes)	2	0.2050	0.1852

Both pruned trees have higher training error than the unpruned tree. Specifically, the unpruned tree has training error 0.1588, the 7-node pruned tree has training error 0.1625,

and the 2-node pruned tree has training error 0.2050. That is what we expect, because more aggressive pruning removes splits and therefore fits the training data less closely.

Problem 3 (k)

Problem 3 (k)

Compare the test error rates between the pruned and unpruned trees. Which is higher?

On the test set, the unpruned tree has error 0.1704, the 7-node pruned tree has error 0.1630, and the 2-node pruned tree has error 0.1852. So the 7-node pruned tree gives the lowest test error, while the 2-node tree pays a noticeable predictive cost for its extra simplicity. This illustrates the tradeoff clearly: the 2-node tree is most parsimonious, but the 7-node tree balances parsimony and predictive performance better on this split.